

P1 — src/wilson_cowan — modelo y librería de estímulos

En una frase: es el corazón del proyecto — define el modelo de Wilson-Cowan (dos poblaciones E e I acopladas que oscilan), lo simula igual que el ode45 de MATLAB, y ofrece una librería de estímulos externos pensada para excitar bien el sistema durante la identificación de sus 10 parámetros.

Ubicación y rol dentro del proyecto

El paquete `src/wilson_cowan/` es la pieza fundacional: todo lo demás (Neural ODE, PINN, controlador) consume el modelo y los estímulos que se definen acá. Contiene dos archivos:

- `model.py` — el modelo, la simulación y la librería de estímulos (el 99 % del código).
- `__init__.py` — un simple re-exportador que expone lo principal para importarlo cómodo desde afuera.

La teoría matemática detrás de estas ecuaciones está en el documento **B3** (teoría del modelo de Wilson-Cowan); los conceptos de sistemas dinámicos que se invocan acá (equilibrio, ciclo límite, punto de operación) están desarrollados en **M2** (sistemas dinámicos). Este documento se queda en el *código*: qué hace cada función, qué recibe y qué devuelve.

Archivo `__init__.py`

No tiene lógica. Su única tarea es re-exportar desde `model.py` las clases y funciones de uso frecuente, para que desde afuera se pueda escribir `from wilson_cowan import WilsonCowan, box_pulse` en lugar de tener que apuntar al submódulo `model`. Reexporta: `WilsonCowan`, `WilsonCowanParams`, `sigmoid`, y los diez generadores de estímulos más `plot_results`.

Archivo `model.py`

Estructura del archivo, de arriba hacia abajo:

1. **Parámetros del sistema** (`WilsonCowanParams`, `sigmoid`).
2. **Señales de entrada / estímulos externos** (los diez generadores).
3. **Clase principal** (`WilsonCowan`: `__init__`, `rhs`, `simulate`).
4. **Visualización** (`plot_results`).
5. Un bloque `if __name__ == "__main__"` que reproduce la simulación de referencia del MATLAB.

Las dependencias son `numpy`, `scipy.integrate.solve_ivp` (el integrador) y `scipy.special.expit` (sigmoidea numéricamente estable). `matplotlib` se importa perezosamente dentro de `plot_results` para no exigirlo si solo se simula.

Sección 1 — Parámetros

`WilsonCowanParams` (dataclass)

Firma:

```
@dataclass
class WilsonCowanParams:
    te=1.0, ti=2.0,
    wEE=6.4, wEI=4.8, wIE=6.0, wII=1.2,
    ae=1.2, ai=1.0,
    thetae=2.8, thetai=4.0
```

Qué hace. Es un contenedor (dataclass) de los **10 parámetros** que definen por completo el comportamiento del modelo. Son exactamente los valores del simulador de MATLAB, elegidos para que el sistema genere oscilaciones

sostenidas (un ciclo límite). Estos 10 números son, precisamente, lo que el proyecto busca **recuperar desde los datos** con Neural ODE y PINN.

Los 10 parámetros:

Parámetro	Símbolo	Valor MATLAB	Qué controla
te	τ_E	1.0	Constante de tiempo de E [s]: qué tan rápido reacciona la población excitatoria (más grande = más lenta).
ti	τ_I	2.0	Constante de tiempo de I [s]: idem para la inhibitoria.
wEE	w_{EE}	6.4	Peso de E sobre sí misma (autoexcitación).
wEI	w_{EI}	4.8	Peso de I sobre E (cuánto la frena I).
wIE	w_{IE}	6.0	Peso de E sobre I (cuánto la activa E).
wII	w_{II}	1.2	Peso de I sobre sí misma (autoinhibición).
ae	a_E	1.2	Ganancia de la sigmoidea de E (qué tan abrupta es la curva en S).
ai	a_I	1.0	Ganancia de la sigmoidea de I.
thetae	θ_E	2.8	Umbral de la sigmoidea de E (a partir de qué entrada dispara con fuerza).
thetai	θ_I	4.0	Umbral de la sigmoidea de I.

Entradas. Los 10 valores; todos tienen default (los del MATLAB), así que `WilsonCowanParams()` ya devuelve el sistema de referencia. Se puede sobrescribir cualquiera, p. ej. `WilsonCowanParams(wEE=5.0)`.

Qué devuelve. Una instancia inmutable-por-defecto con los 10 campos accesibles como atributos.

Propiedades derivadas: ke y ki Firmas:

```
@property
def ke(self) -> float: return 1.0 / (1.0 + np.exp(self.ae * self.thetae))
@property
def ki(self) -> float: return 1.0 / (1.0 + np.exp(self.ai * self.thetai))
```

Qué son y por qué existen. No son parámetros libres: se **calculan** a partir de los otros. `ke` es el valor de la sigmoidea excitatoria evaluada en reposo (entrada total = 0), y `ki` lo mismo para la inhibitoria. Se **restan** dentro de la ecuación de cada población para forzar que, sin estímulo externo, el punto **E = I = 0 sea un equilibrio** del sistema — que el modelo “duerma” en cero cuando nadie lo excita.

La lógica: en la ecuación de E aparece el término `sigmoid(u_e) - ke`. En reposo (todo cero salvo el umbral), `sigmoid(-theta_e...)` vale exactamente `ke`, así que el paréntesis se anula y $dE/dt = 0$. Sin este offset, el reposo del modelo no estaría en cero y sería incómodo trabajar con él. Como `ke`, `ki` se recalculan solas si cambian `ae/ai/thetae/thetai`, el reposo en cero se mantiene siempre. (Ver **M2** para el concepto de equilibrio.)

sigmoid

Firma: `sigmoid(u: np.ndarray | float, a: float) -> np.ndarray | float`

Qué hace. Es la función de activación en forma de S del modelo: $S(u) = \frac{1}{1 + e^{-au}}$. Convierte la entrada total u de una población en su tasa de disparo, un número entre 0 y 1.

Entradas. u (entrada total, escalar o array) y a (ganancia, que controla lo abrupto de la curva).

Qué devuelve. El valor de la sigmoidea, mismo shape que u .

Notas. Internamente usa `scipy.special.expit(a*u)`, que es la misma fórmula pero numéricamente estable (no se desborda con argumentos grandes). Importante: el **umbral no entra acá** — viene ya restado dentro de u en la ecuación, igual que en el MATLAB.

Sección 2 — Estímulos externos $P(t)$ y $Q(t)$

Idea general. Son las señales que se inyectan al sistema desde afuera: $P(t)$ entra a la población E y $Q(t)$ a la I . Sin estímulo el sistema queda quieto en reposo; al prender un estímulo, arranca a oscilar. Todos los generadores son **fábricas de funciones**: reciben parámetros de forma y devuelven una función $f(t)$ que da el valor del estímulo en cada instante.

Por qué hay tantos (clave para la identificación). Un pulso cuadrado es casi “DC”: excita el sistema por un rango pobre de estados y deja el problema inverso (recuperar los parámetros) mal condicionado. Para identificar bien un sistema **no lineal** hace falta recorrer muchas **amplitudes** (la sigmoidea responde distinto en cada punto de operación) y muchas **frecuencias** — es el principio de *excitación persistente* (Ljung). Distintos estímulos barren mejor ese espacio.

Dos detalles transversales a destacar:

- **(a) Las señales aleatorias PRE-CALCULAN su “agenda”.** El integrador `solve_ivp` llama a la entrada en tiempos saltados, repetidos y fuera de orden. Si una señal aleatoria sorteara su valor en cada llamada, $f(t)$ no sería reproducible ni consistente. Por eso los generadores aleatorios (`aprb_pulse`, `prbs_pulse`, `poisson_pulse`) sortean de antemano —a partir de una **semilla seed**— cuándo cambian y a qué valor, guardando esos bordes/tiempos en arrays. Luego $f(t)$ solo **consulta** en qué tramo cae t . Así $f(t)$ es una **función pura del tiempo**: mismo $t \rightarrow$ mismo valor siempre, y toda la señal es reproducible fijando la semilla.
- **(b) On/off y ≥ 0 = realizables con optogenética.** Varios estímulos son binarios (prenden/apagan) y todos los “tipo pulso” son ≥ 0 . Eso importa porque la actuación real por **optogenética** solo puede *prender o apagar* luz: no puede inyectar amplitudes negativas ni señales suaves arbitrarias. Los estímulos on/off y no negativos son entonces físicamente realizables en el laboratorio; las senoides suaves, no.

Sobre las unidades de frecuencia. Las frecuencias van en las mismas unidades de tiempo que la simulación (segundos acá; t hasta ~ 600 , $t_e=1$, $t_i=2$), **no** son los Hz biológicos ($\gamma \sim 40$ Hz, $\theta \sim 6$ Hz): a esa velocidad el sistema no responde. Lo que se conserva es la *relación* entre frecuencias ($\gamma/\theta \sim 6:1$) y que caigan en la banda donde el sistema reacciona ($\sim 0.005\text{--}0.05$ en estas unidades).

Los estímulos se dividen en tres familias.

Familia 1 — Básicos

box_pulse **Firma:** `box_pulse(amplitude, t_on, t_off) -> Callable[[float], float]`

Qué hace / qué señal genera. Un pulso cuadrado: vale `amplitude` mientras $t_{\text{on}} \leq t < t_{\text{off}}$, y 0 fuera. Es el estímulo de referencia del MATLAB ($P = \text{box_pulse}(0.8, 100, 400)$, $Q = \text{box_pulse}(0.6, 200, 500)$).

Entradas. `amplitude` (altura), `t_on` (instante de prendido), `t_off` (instante de apagado).

Qué devuelve. La función $f(t)$.

Para qué sirve en identificación. Es el caso base: reproduce la simulación original y sirve de sanity check. Pero excita pobremente (casi DC), así que para identificar los parámetros con precisión se prefieren los estímulos ricos de abajo.

zero_input **Firma:** zero_input(t) -> float (devuelve siempre 0.0)

Qué hace. Entrada nula. Es el default de P y Q en WilsonCowan: sirve para simular el sistema libre, sin ningún estímulo externo (verifica que el reposo $E=I=0$ realmente se sostiene).

Notas. A diferencia de los demás, no es una fábrica: ya es la función $f(t)$.

Familia 2 — Señales que varían en el tiempo (legacy)

Estas tres varían poco la amplitud y **no** son on/off, así que **no** son realizables con optogenética. Se conservan como legado (el MATLAB ya las soportaba) pero no son la vía preferida del proyecto. Por defecto llevan `offset = amplitud` para que la señal quede ≥ 0 .

sine_pulse (legacy) **Firma:** sine_pulse(amplitud, freq, t_on, t_off, offset=None) -> Callable

Qué señal genera. Una senoide de una sola frecuencia, gateada a la ventana: `offset + amplitud * sin(2 $\pi$  * freq * (t - t_on))`.

Entradas. amplitud, freq (frecuencia), ventana t_on/t_off, offset (default = amplitud, para que quede ≥ 0).

Qué devuelve. $f(t)$.

Para qué sirve. Excita una única frecuencia. Útil para respuesta en frecuencia puntual, pero pobre para identificación (una sola frecuencia, amplitud casi constante).

multisine_pulse (legacy) **Firma:** multisine_pulse(amplitud, freqs, t_on, t_off, offset=None) -> Callable

Qué señal genera. Suma de senoides de varias frecuencias (freqs es una lista). La amplitud total se reparte entre las frecuencias ($a = \text{amplitud}/\text{len}(\text{freqs})$).

Entradas. amplitud, freqs (iterable de frecuencias), ventana, offset.

Qué devuelve. $f(t)$.

Para qué sirve. Excitación más rica que la senoide simple: cubre varias frecuencias a la vez. Aun así no barre amplitud ni es on/off.

chirp_pulse **Firma:** chirp_pulse(amplitud, f0, f1, t_on, t_off, offset=None) -> Callable

Qué señal genera. Un barrido lineal de frecuencia de f_0 a f_1 a lo largo de la ventana (fase con término cuadrático $f_0 \cdot \tau + \frac{1}{2} \cdot k \cdot \tau^2$, $k = (f_1 - f_0)/\text{duración}$).

Entradas. amplitud, f0 (frecuencia inicial), f1 (final), ventana, offset.

Qué devuelve. $f(t)$.

Para qué sirve. Excita un **rango continuo** de frecuencias en una sola corrida — muy bueno para identificabilidad en el eje frecuencia. Su límite: sigue sin barrer amplitud y no es on/off.

Familia 3 — Tipo pulso / escalón (on-off, realizables con optogenética)

Todos ≥ 0 y conmutan (prenden/apagan). Barren amplitud y/o frecuencia mucho mejor que las senoïdes, y son **físicamente realizables con optogenética**. Los aleatorios usan seed.

aprbbs_pulse — el caballo de batalla **Firma:** aprbbs_pulse(amplitud, t_on, t_off, dwell_min, dwell_max, seed=0, amp_min=0.0) -> Callable

Qué señal genera. Escalones de **amplitud Y duración aleatorias** (Amplitude-modulated PRBS). En cada tramo sortea una duración en `[dwell_min, dwell_max]` y una amplitud en `[amp_min, amplitud]`. Pre-calcula la agenda de bordes y valores con `default_rng(seed)`; $f(t)$ ubica el tramo con `searchsorted`.

Entradas. amplitude (tope de amplitud), ventana, dwell_min/dwell_max (rango de duración de cada escalón), seed, amp_min (piso de amplitud: con 0 la señal se “apaga” seguido; subirlo la mantiene viva sin perder el barrido).

Qué devuelve. $f(t)$.

Para qué sirve. El **estímulo más importante para identificación no lineal**: cubre a la vez el eje **amplitud** y el eje **frecuencia** (duraciones variables). Recorre muchos puntos de operación de la sigmoidea, que es justo lo que hace identificable un sistema no lineal.

theta_gamma_pulse — el régimen propio del proyecto Firma: theta_gamma_pulse(amplitude, f_gamma, f_theta, t_on, t_off, duty=0.5) -> Callable

Qué señal genera. Ráfagas de pulsos rápidos (tren **gamma**, on/off según duty) cuya intensidad la modula una envolvente lenta (**theta**, $\text{env} = \frac{1}{2}(1 + \sin(2\pi \cdot f_{\text{theta}} \cdot t))$ en $[0,1]$). Valor = amplitude · env · gamma_on.

Entradas. amplitude, f_gamma (frecuencia del tren rápido), f_theta (envolvente lenta), ventana, duty (fracción del período gamma que está prendido).

Qué devuelve. $f(t)$.

Para qué sirve. Es el **régimen propio del proyecto**: reproduce el acoplamiento theta-gamma que el controlador busca inducir. Sirve para identificar y validar el modelo en el mismo régimen en el que después va a operar. No usa aleatoriedad (es determinístico).

square_wave_pulse Firma: square_wave_pulse(amplitude, freq, t_on, t_off, duty=0.5) -> Callable

Qué señal genera. Onda cuadrada / tren de pulsos periódico: prende/apaga a frecuencia freq, con duty como fracción del período que está prendido.

Entradas. amplitude, freq, ventana, duty.

Qué devuelve. $f(t)$.

Para qué sirve. El estímulo estándar en **DBS** (estimulación cerebral profunda) y optogenética. Periódico y binario, buen sanity check de respuesta a un tren regular. Determinístico.

prbs_pulse Firma: prbs_pulse(amplitude, t_on, t_off, bit_period, seed=0) -> Callable

Qué señal genera. Secuencia binaria pseudo-aleatoria: cada bit_period el valor es 0 o amplitude al azar. Sorteando todos los bits de antemano con default_rng(seed).

Entradas. amplitude, ventana, bit_period (duración de cada bit), seed.

Qué devuelve. $f(t)$.

Para qué sirve. Clásico de identificación (Ljung): espectro de banda ancha, y por ser binario puro es **optogenética directa**. Limitación: una sola amplitud (no barre el eje amplitud como APRBS), por eso suele ir como **complemento** del APRBS.

poisson_pulse Firma: poisson_pulse(amplitude, rate, t_on, t_off, pulse_width, seed=0) -> Callable

Qué señal genera. Tren de pulsos cortos (ancho pulse_width) en tiempos aleatorios, con rate eventos por unidad de tiempo (inter-arribos exponenciales). Pre-calcula los instantes de evento con default_rng(seed); $f(t)$ prende si t cae dentro del ancho de pulso del último evento.

Entradas. amplitude, rate (tasa de eventos), ventana, pulse_width (ancho de cada pulso), seed.

Qué devuelve. $f(t)$.

Para qué sirve. Es el estímulo **naturalista**: imita la estadística de disparos neuronales reales (proceso de Poisson). Útil para validar el modelo bajo entradas con estructura temporal parecida a la biológica.

Tabla resumen de estímulos

Generador	Qué es	Para qué (en identificación)
box_pulse	Pulso cuadrado (una ventana)	Caso base / referencia del MATLAB; excita pobre.
zero_input	Cero constante	Sistema libre; verificar el reposo.
sine_pulse (<i>legacy</i>)	Senoide de una frecuencia	Frecuencia puntual; no on/off.
multisine_pulse (<i>legacy</i>)	Suma de senoides	Varias frecuencias a la vez; no on/off.
chirp_pulse	Barrido lineal de frecuencia	Rango continuo de frecuencias; no on/off.
aprbps_pulse	Escalones de amplitud y duración aleatorias	Caballo de batalla: barre amplitud × frecuencia. Optogenética.
theta_gamma_pulse	Ráfagas gamma moduladas por envolvente theta	Régimen propio del proyecto; identificar/validar en operación.
square_wave_pulse	Tren de pulsos periódico (duty)	Estándar de DBS/optogenética.
prbs_pulse	Secuencia binaria pseudo-aleatoria	Banda ancha (Ljung); optogenética directa; complementa APRBS.
poisson_pulse	Pulsos cortos en tiempos de Poisson	Naturalista: estadística de disparos reales.

Sección 3 y 4 — Clase principal WilsonCowan

Junta parámetros + entradas + derivada + integración en un solo objeto.

WilsonCowan.__init__

Firma: __init__(self, params=None, P=zero_input, Q=zero_input) -> None

Qué hace. Construye el modelo. Guarda los parámetros y las dos funciones de entrada externa.

Entradas. params (un WilsonCowanParams; si es None usa los del MATLAB), P (función de entrada a E, default zero_input), Q (función de entrada a I, default zero_input).

Qué devuelve. Nada (inicializa self.params, self.P, self.Q).

WilsonCowan.rhs — la función derivada (el corazón)

Firma: rhs(self, t: float, state: np.ndarray) -> list[float]

Qué hace. Dado el instante t y el estado actual [I, E], devuelve la velocidad de cambio [dI/dt, dE/dt]. El integrador la llama muchísimas veces para avanzar la solución paso a paso.

Primero calcula la **entrada total** que recibe cada población (influencias internas + estímulo externo – umbral):

$u_i = w_{IE} \cdot E - w_{II} \cdot I + Q(t) - \theta_{Ii}$
 $u_e = w_{EE} \cdot E - w_{EI} \cdot I + P(t) - \theta_{Ee}$

Y luego **las dos ecuaciones del modelo:**

$$\frac{dI}{dt} = \frac{1}{\tau_I} \left(-I + S_i(w_{IE}E - w_{II}I + Q(t) - \theta_I) - k_i \right)$$

$$\frac{dE}{dt} = \frac{1}{\tau_E} \left(-E + S_e(w_{EE}E - w_{EI}I + P(t) - \theta_E) - k_e \right)$$

Cada población tiende a **relajarse** (términos $-I$, $-E$) y a la vez es **empujada** por su sigmoidea (menos el offset de reposo k_i/k_e), todo escalado por $1/\text{constante_de_tiempo}$.

Entradas. t (instante), $\text{state} = [I, E]$ (¡en ese orden — I primero!).

Qué devuelve. La lista $[dI, dE]$.

Notas. El orden del estado es $[I, E]$ (fiel al MATLAB), fácil de confundir. La salida de interés del modelo no es I ni E por separado sino el "potencial" $y = E - I$, que se calcula en `simulate`. (Para el sentido dinámico de estas ecuaciones —ciclo límite, equilibrio— ver **B3** y **M2**.)

WilsonCowan.simulate — integración numérica

Firma:

```
simulate(self, I0=0.0, E0=0.0, t_span=(0.0, 600.0), t_eval=None,
         rel_tol=1e-3, abs_tol=1e-6, method="RK45") -> dict[str, np.ndarray]
```

Qué hace. Avanza el sistema desde t_{inicial} hasta t_{final} partiendo de $[I_0, E_0]$, delegando en `scipy.integrate.solve_ivp`. Es el equivalente del `ode45` de MATLAB: **RK45** es Runge-Kutta 4(5) de paso adaptativo, y las tolerancias por defecto ($\text{rtol}=1e-3$, $\text{atol}=1e-6$) son las mismas que las de MATLAB. Después reconstruye las entradas $P(t)$, $Q(t)$ en los tiempos resueltos para poder graficarlas.

Entradas. I_0 , E_0 (condiciones iniciales), t_{span} (par (inicio, fin)), t_{eval} (grilla de tiempos donde devolver la solución; si es `None`, devuelve los puntos que el propio integrador elige con su paso adaptativo), rel_tol , abs_tol (tolerancias), `method` (integrador; "RK45" $\approx$ `ode45`).

Qué devuelve. Un diccionario con arrays: "t", "I", "E", "y" ($= E - I$, la salida final), "P" y "Q" (las entradas reconstruidas en t).

Notas. $y = E - I$ es la magnitud que después se compara contra los datos en la identificación. Como t_{eval} puede fijar una grilla uniforme, es directo alinear la salida con datos experimentales.

Sección 5 — Visualización

plot_results

Firma: `plot_results(sol, save_path="results/figures/simulacion_wilson_cowan.png", show=False)`

Qué hace. Toma el diccionario que devuelve `simulate()` y arma una figura de **3 paneles** (mismo layout que el MATLAB, compartiendo el eje del tiempo):

1. **Entradas:** las señales externas P (a E) y Q (a I).
2. **Estados:** la actividad I (inhibitoria) y E (excitatoria).
3. **Salida:** el potencial $y = E - I$.

Entradas. `sol` (el dict de `simulate`), `save_path` (dónde guardar el PNG; crea la carpeta si no existe; `None` para no guardar), `show` (si `True`, además muestra la figura en pantalla).

Qué devuelve. El objeto `fig` de `matplotlib`.

Notas. Importa `matplotlib` perezosamente adentro de la función, para no exigir la dependencia si solo se simula sin graficar. Guarda a 150 dpi.

Bloque de ejecución directa

Al final, `if __name__ == "__main__":` reproduce la **simulación de referencia del MATLAB**: crea un Wilson-Cowan con los parámetros por defecto y $P = \text{box_pulse}(0.8, 100, 400)$, $Q = \text{box_pulse}(0.6, 200, 500)$; integra de 0 a 600 s sobre una grilla de 6000 puntos; y guarda la figura de 3 paneles. Se corre con `python -m src.wilson_cowan.model`.

Ver también

- **B3** — teoría del modelo de Wilson-Cowan (de dónde salen las ecuaciones dE/dt y dI/dt).
- **M2** — sistemas dinámicos (equilibrio, ciclo límite, punto de operación, excitación persistente).